Model Quality

Software Engineering for Intelligent Systems

JOÃO EDUARDO MONTANDON

How to measure

quality?

Quality comes in different flavors...

- Are required **features** implemented?
- Does it **operate** correctly under expected conditions?
- Does the implementation follow the intended **architecture**?

Quality comes in different flavors...

- Are required **features** implemented?
  (Acceptance tests, test coverage)
- Does it **operate** correctly under expected conditions?
  (Uptime period, Stress testing)
- Does the implementation follow the intended **architecture**?
  (Static analysis, architectural reviews)

If we care about something, it must be detectable.

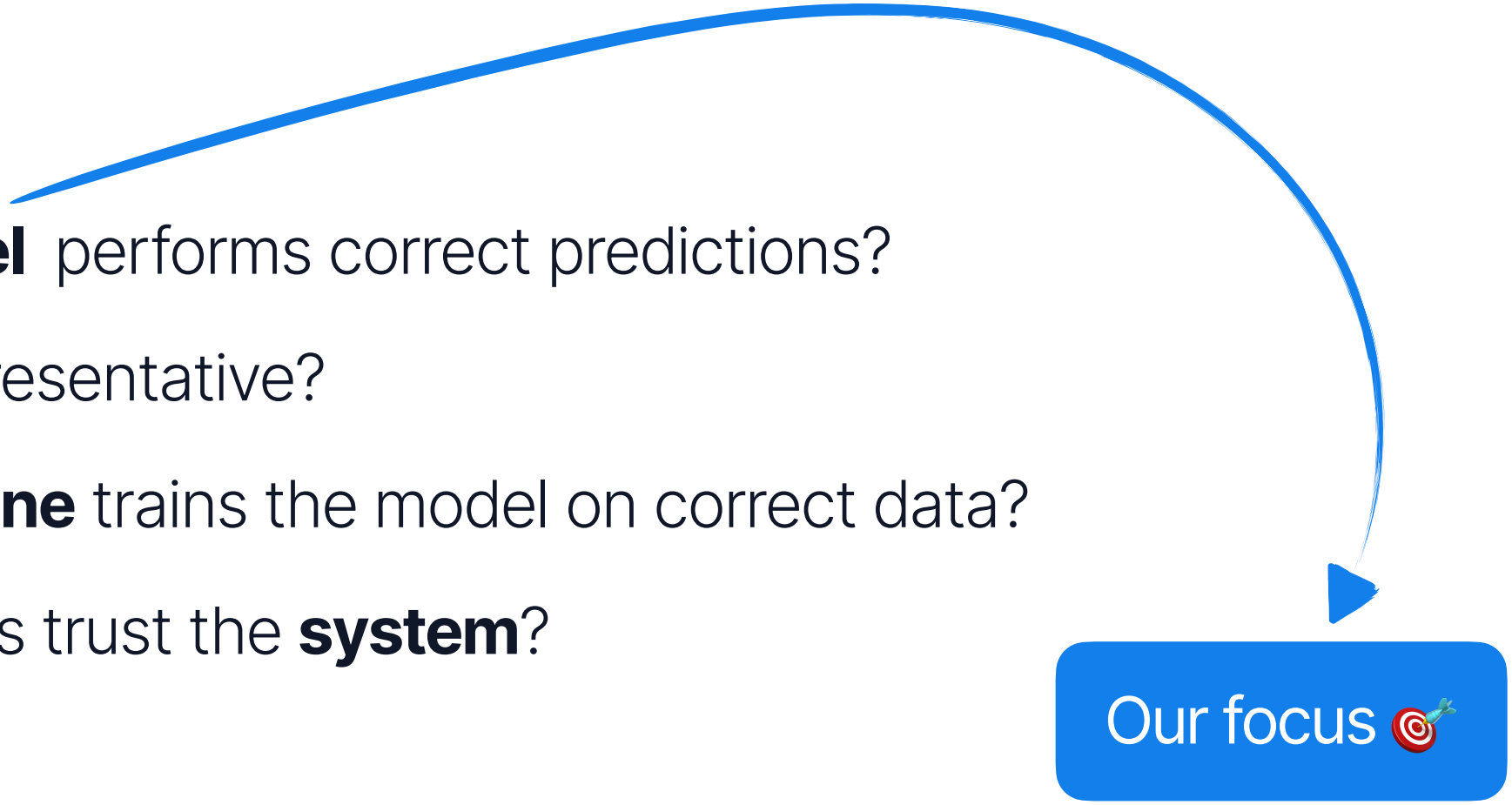
And we care about our models... right?

Intelligent Systems bring a lot of new concerns!

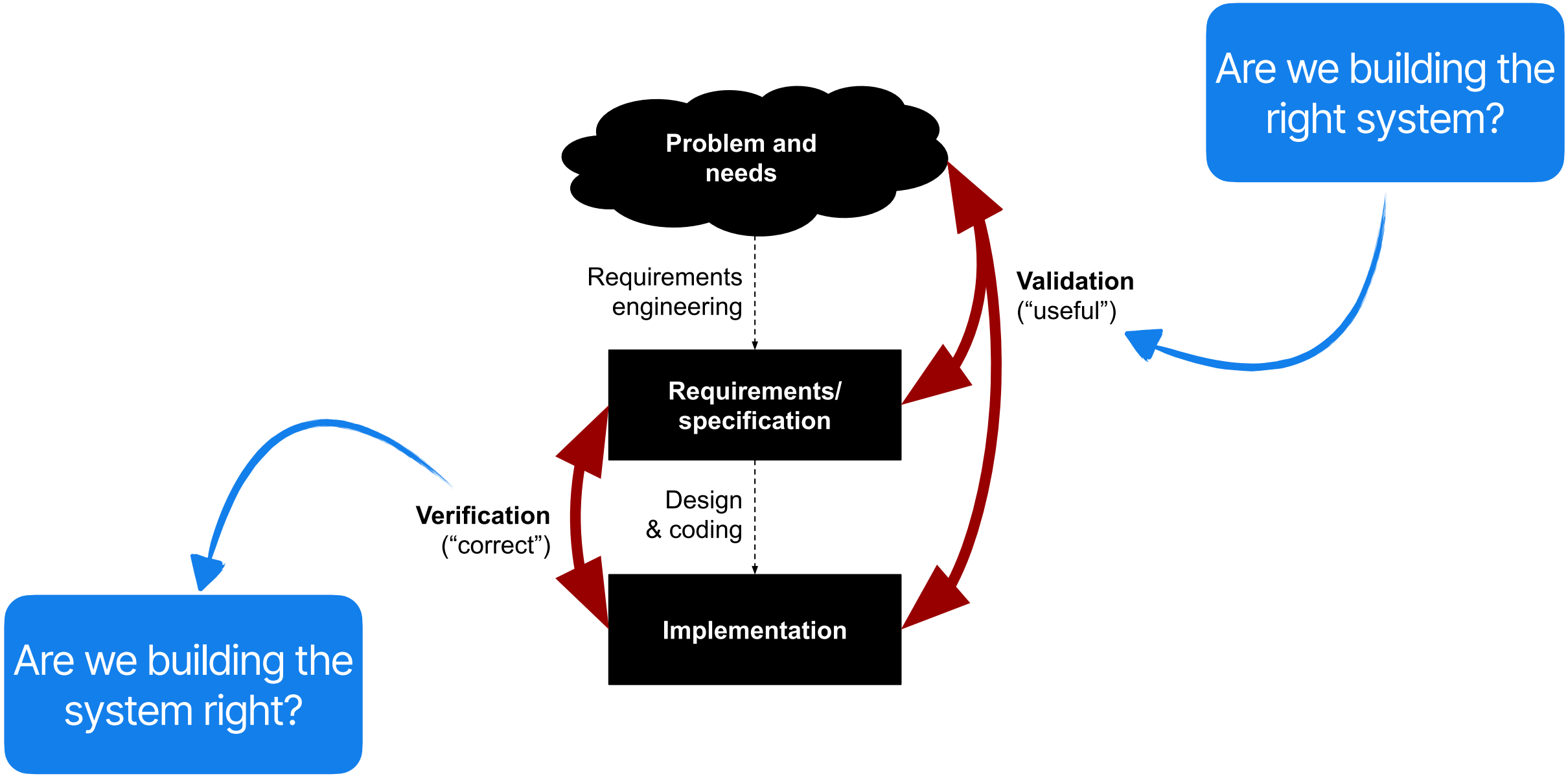
- Does the **model** performs correct predictions?
- Is the **data** representative?
- Does the **pipeline** trains the model on correct data?
- Do stakeholders trust the **system**?

Intelligent Systems bring a lot of new concerns!

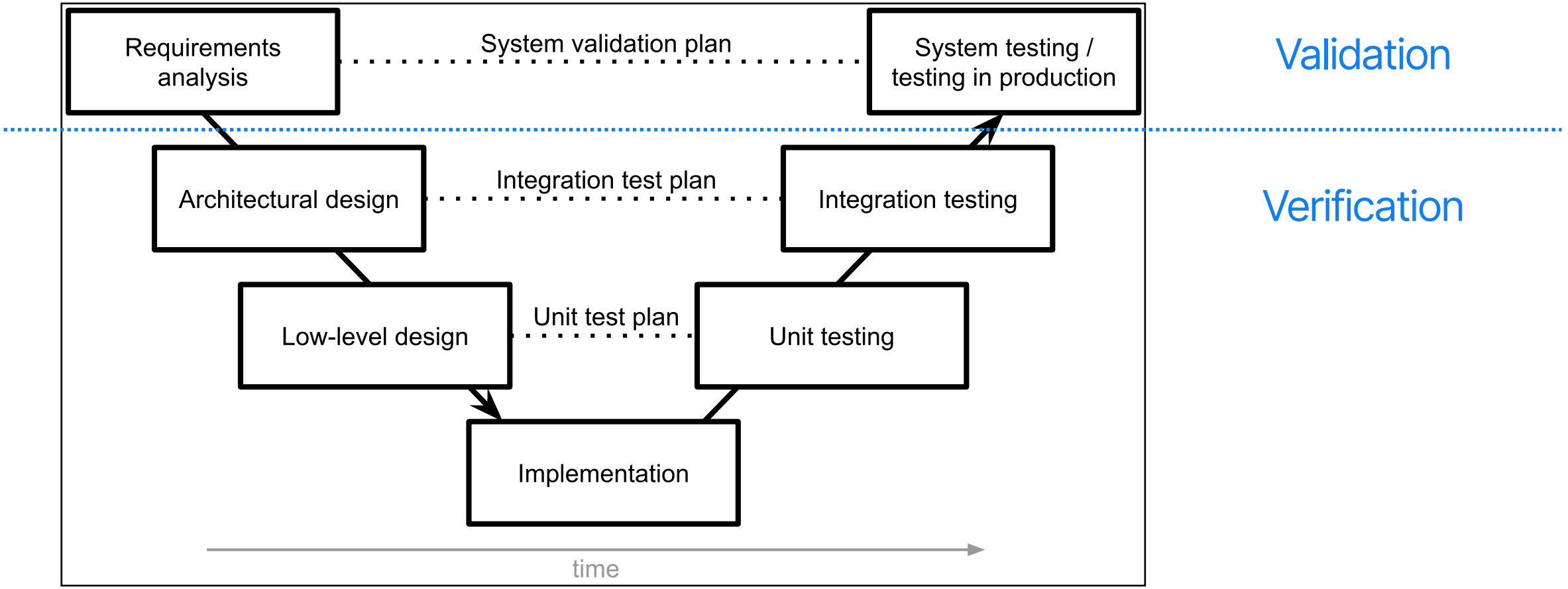
- Does the **model** performs correct predictions?
- Is the **data** representative?
- Does the **pipeline** trains the model on correct data?
- Do stakeholders trust the **system**?



What is [model] correctness?


Are we building the right system?
Problem and needs
Requirements engineering
Validation ("useful")
Requirements/ specification
Verification ("correct")
Design & coding
Implementation
Are we building the system right?

Correct with regard to what?



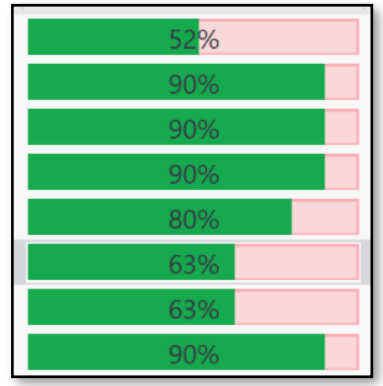
The verification lifecycle

```python
def next_date(year, month, day):
    """Spec: Given a year, a month (range 1-12), and a day (1-31),
    the function returns the date of the following calendar day
    in the Gregorian calendar as a triple of year, month, and day.
    Throws InvalidInputException for inputs that are not valid dates.
    """
```

```python
def test_next_date_simple():
    assertTrue(Date(2020,01,01).next() == Date(2020,01, 02))

def test_next_date_end_of_31_day_month():
    ...

def test_next_date_end_of_30_day_month():
    ...
```



$ pytest date.py

90% are implemented **correctly**

```python
def hasCancer(image: Image, age: int, ...) -> bool:
    """Determine whether the radiology image shows signs of cancer
    ?????
    ?????
    """
```

No specification...

```python
def test_scenario1(self):
    ?????

def test_scenario2(self):
    ?????
```

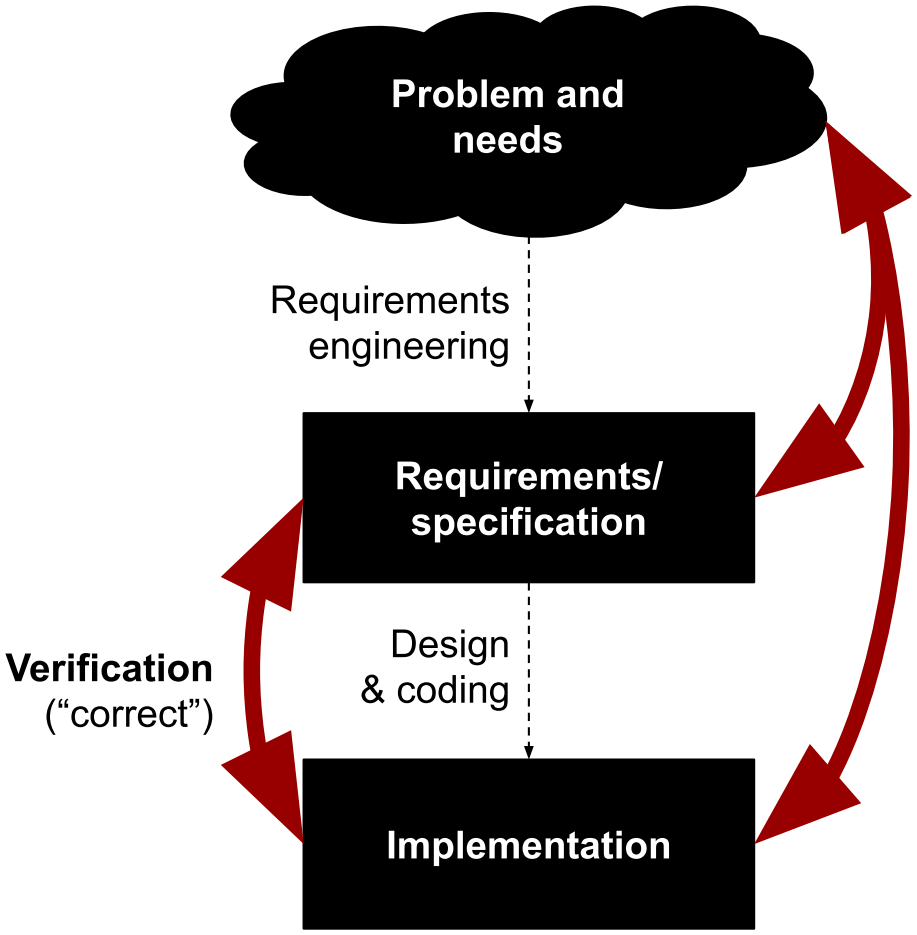
No tests....

$????

How much have I done? 🤔

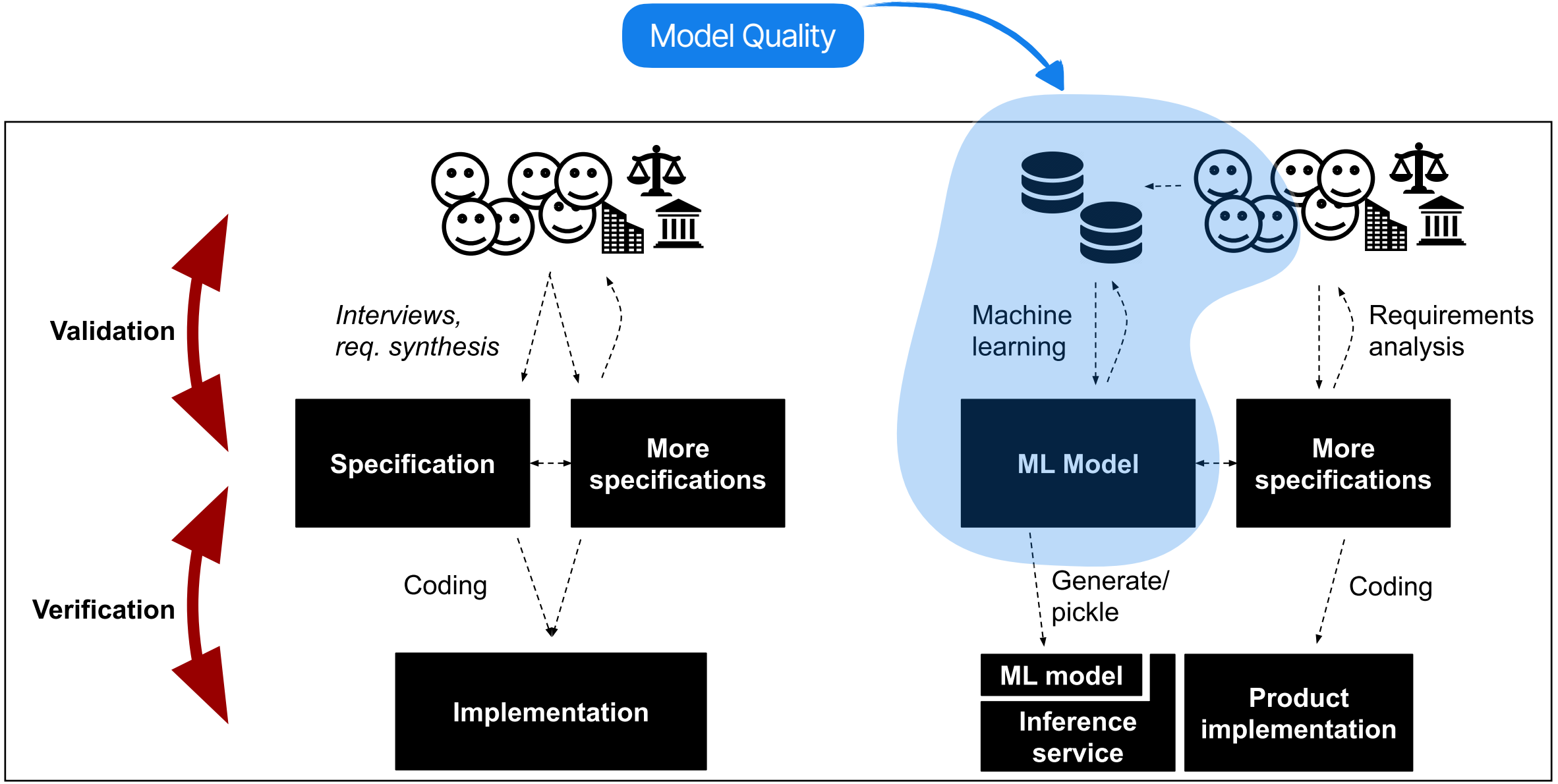
> All models are wrong, but some are useful. So the question you need to ask is not 'Is the model true?' (It never is) but '**Is the model good enough for this particular application?**'"

— George Box



Are we building the **right model?**

We do not expect the models to be "correct" [...], but we looking for models that are **useful** for a given problem.


Model Quality
Validation
Verification
Interviews,
req. synthesis
Specification
More
specifications
Coding
Implementation
Machine
learning
Requirements
analysis
ML Model
More
specifications
Generate/
pickle
Coding
ML model
Inference
service
Product
implementation

On Terminology

- Avoid "bug", since "bug" refers to requirement implemented incorrectly.
- Avoid "correctness", favor "fit" or “accuracy”.
- In ML-world, $performance \equiv accuracy$.
  We will use accuracy.

Accuracy Measures

Model Accuracy

- The most common way to measure a model quality level.
- Model with higher accuracy is better than a model if a lower one.

There are tons of metrics

Our goal is to understand **limitations** and **pitfalls**

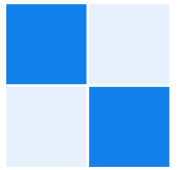
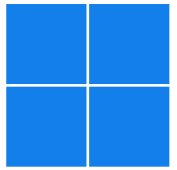
Classification	Accuracy, Precision, Recall, F1-score, ROC AUC, PR AUC, Log Loss, MCC
Regression	MSE, RMSE, MAE, MAPE, R^2, Adjusted R^2
NLP	Perplexity, BLEU, ROUGE, METEOR, BERTScore, Exact Match, Token-level F1, Recall@k, MRR, NDCG

Remembering Confusion Matrix

		Actual	
		Positive	**Negative**
Predicted	**Positive**	TP	FP
	Negative	FN	TN

Accuracy

What is the proportion of correct classifications?

$$accuracy = \frac{correct\ classifications}{total\ classifications} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\text{  }}{\text{  }}$$

https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall

Accuracy

"Our ML model has achieved an accuracy of 0.97 for cancer detection". **How good is this?** 🤔

	Positive	Negative
Positive	??	??
Negative	??	??

Accuracy

$$\frac{4 + 9670}{4 + 325 + 1 + 9670} = 0.97$$

"Our ML model has achieved an accuracy of 0.97 for cancer detection". **How good is this?** 🤔

	Positive	Negative
Positive	4	325
Negative	1	9670

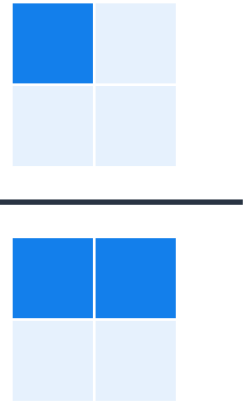
> “Since all models are wrong, the scientist must be alert to **what is importantly wrong**.”
>
> — George Box

Both FN and FP have costs, but they are not necessarily equal.
Which one to minimize?

Precision

What proportion of positive detections was actually correct?

$$precision = \frac{correctly\ classified\ actual\ positives}{everything\ classified\ as\ positive} = \frac{TP}{TP + FP} =$$



https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall

Recall

What is the proportion of actual positives?

$$recall = \frac{correcly\ classified\ actual\ positives}{all\ actual\ positives} = \frac{TP}{TP + FN} = \frac{\blacksquare\ \square}{\blacksquare\ \square}$$

https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall

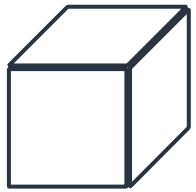
$$prec = \frac{4}{4 + 325} = 0.01 \qquad rec = \frac{4}{4 + 1} = 0.80 \qquad acc = \frac{4 + 9670}{4 + 325 + 1 + 9670} = 0.97$$

	Positive	Negative
Positive	4	325
Negative	1	9670

The importance of baselines

Which **improvement** is better?? 🤔

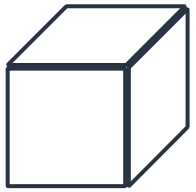
A:



Acc = 50%

Acc = 75%

B:



Acc = 99.9%

Acc = 99.99%

The importance of baselines

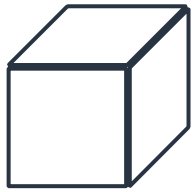
- Accuracy are meaningless if interpreted in isolation.
- There is no idea of progress.

$$error\ reduction = \frac{(1 - baseline) - (1 - model)}{1 - baseline}$$

- **By how much** does the **new** model **reduce mistakes** compared to the **old** one?

$$\frac{(1-0.5)-(1-0.75)}{1-0.5} = \frac{0.25}{0.5} = 50\,\%$$

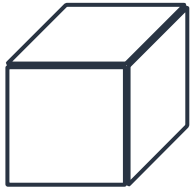
A:



Acc = 50%

Acc = 75%

B:



Acc = 99.9%

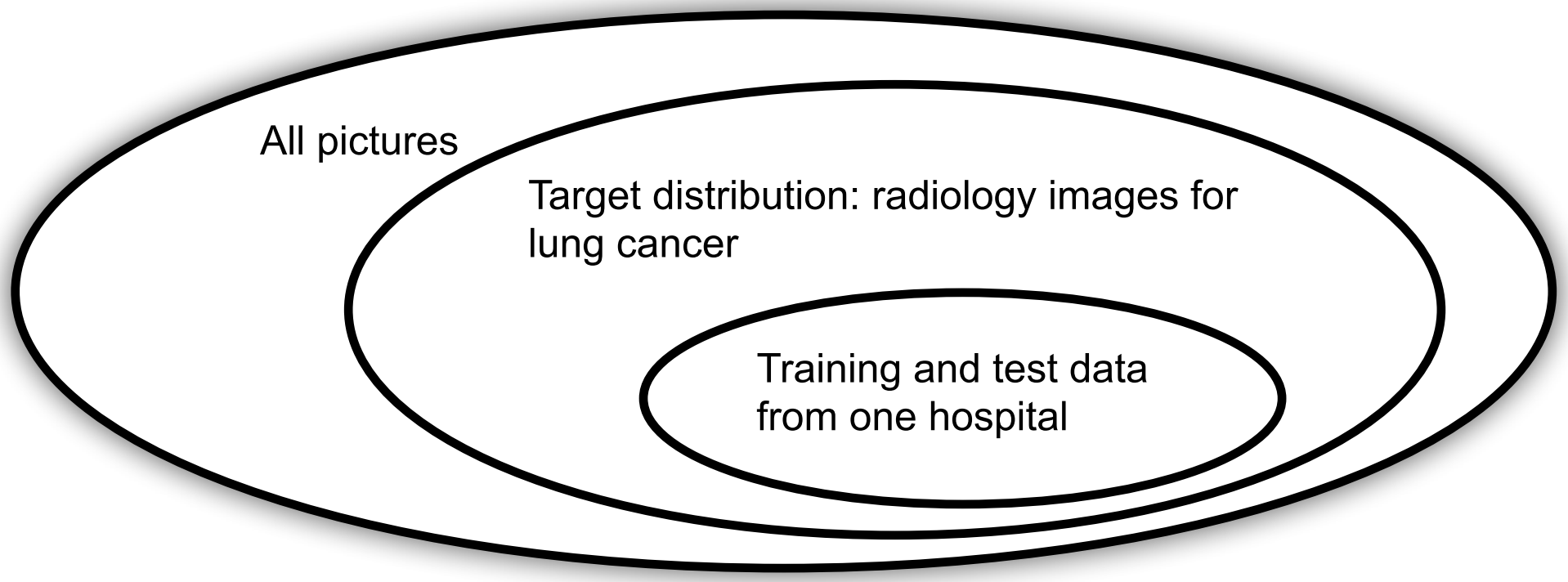
Acc = 99.99%

$$\frac{(1-0.99)-(1-0.9999)}{1-0.99} = \frac{0.0099}{0.01} = 99\,\%$$

Pitfalls

When measuring accuracy

P1. Using test data that are not representative



P2. Using misleading accuracy measures

- **Ignore the context** of your **application** when selecting a measure.
  - Accuracy for cancer detection, makes sense?
  - Report results without baseline?
- Observe **overall results** may overlook some nuances.
  - Cancer consistently fail at specific groups (black women).

P3. Data leakage

- Data can be shared between training and test/validation data in surprisingly ways.
  - Incorrect split
  - Data overlap
  - Unified preprocessing
- Every time a leak happens, the model can overfits.

P3. Example

Chenyang Yang, Rachel A Brower-Sinning, Grace Lewis, and Christian Kastner. 2023. Data Leakage in Notebooks: Static Detection and Better Processes. ASE 2022.

```
1  import numpy as np
2  # generate random data
3  n_samples, n_features, n_classes = 200, 10000, 2
4  rng = np.random.RandomState(42)
5  X = rng.standard_normal((n_samples, n_features))
6  y = rng.choice(n_classes, n_samples)
7
8  # leak test data through feature selection
9  X_selected = SelectKBest(k=25).fit_transform(X, y)
10
11 X_train, X_test, y_train, y_test = train_test_split(
12     X_selected, y, random_state=42)
13 gbc = GradientBoostingClassifier(random_state=1)
14 gbc.fit(X_train, y_train)
15
16 y_pred = gbc.predict(X_test)
17 accuracy_score(y_test, y_pred)
18 # expected accuracy ~0.5; reported accuracy 0.76
```

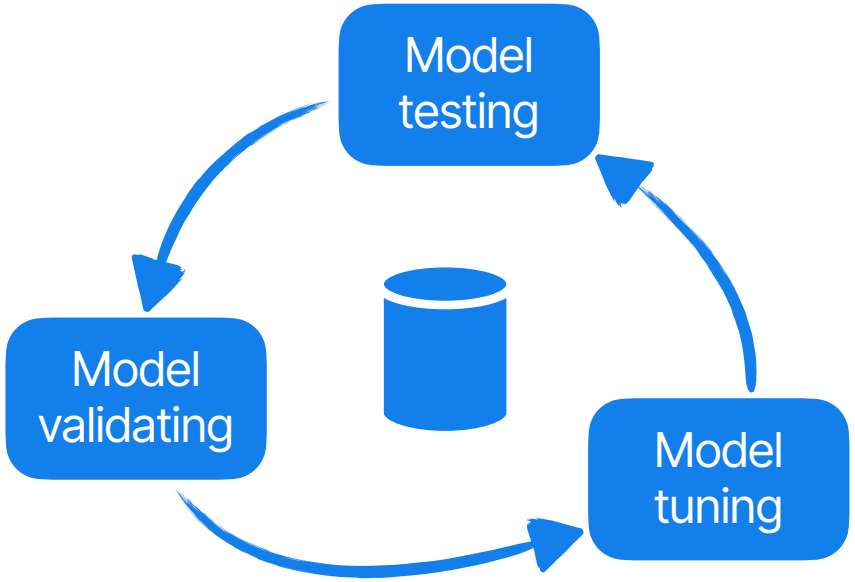
Train + test

Univariate feature selection works by selecting the best features based on univariate statistical tests. It can be seen as a preprocessing step to an estimator. Scikit-learn exposes feature selection routines as objects that implement the `transform` method:

- **SelectKBest** removes all but the k highest scoring features

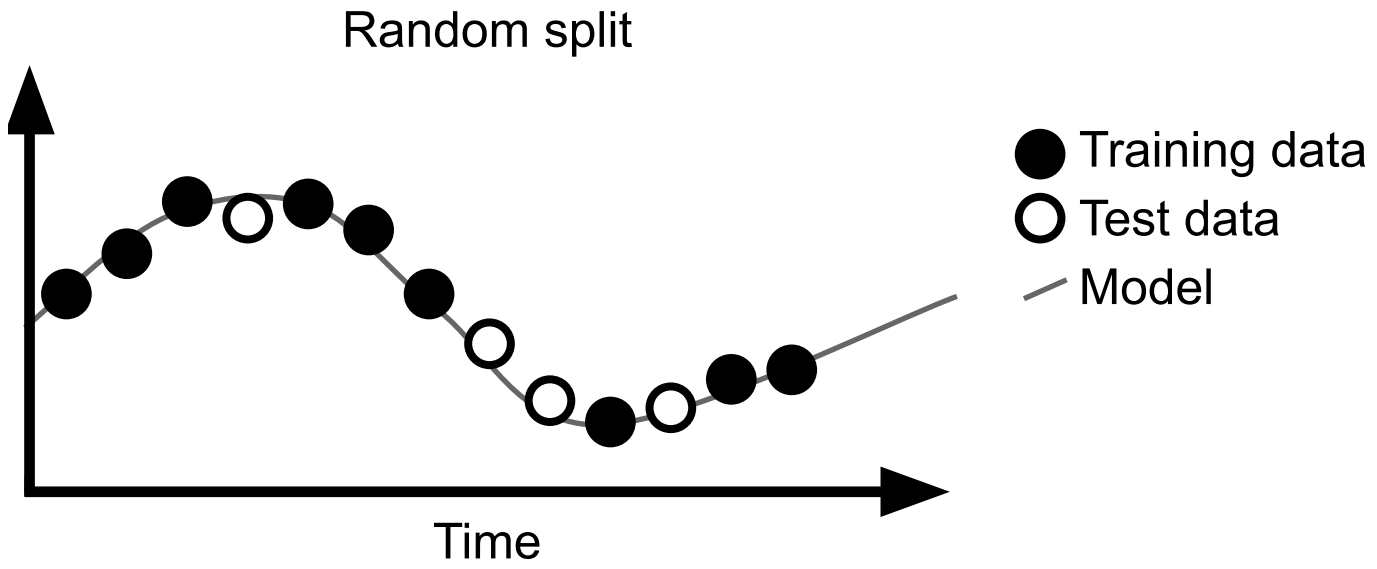
P4. Overfitting from repeated test data

- Happens when developers repeatedly improves the model on the same data.
- The model will increasingly be better only on that specific distribution.
    - a.k.a, overfitting
- Test data should be continuously refreshed.



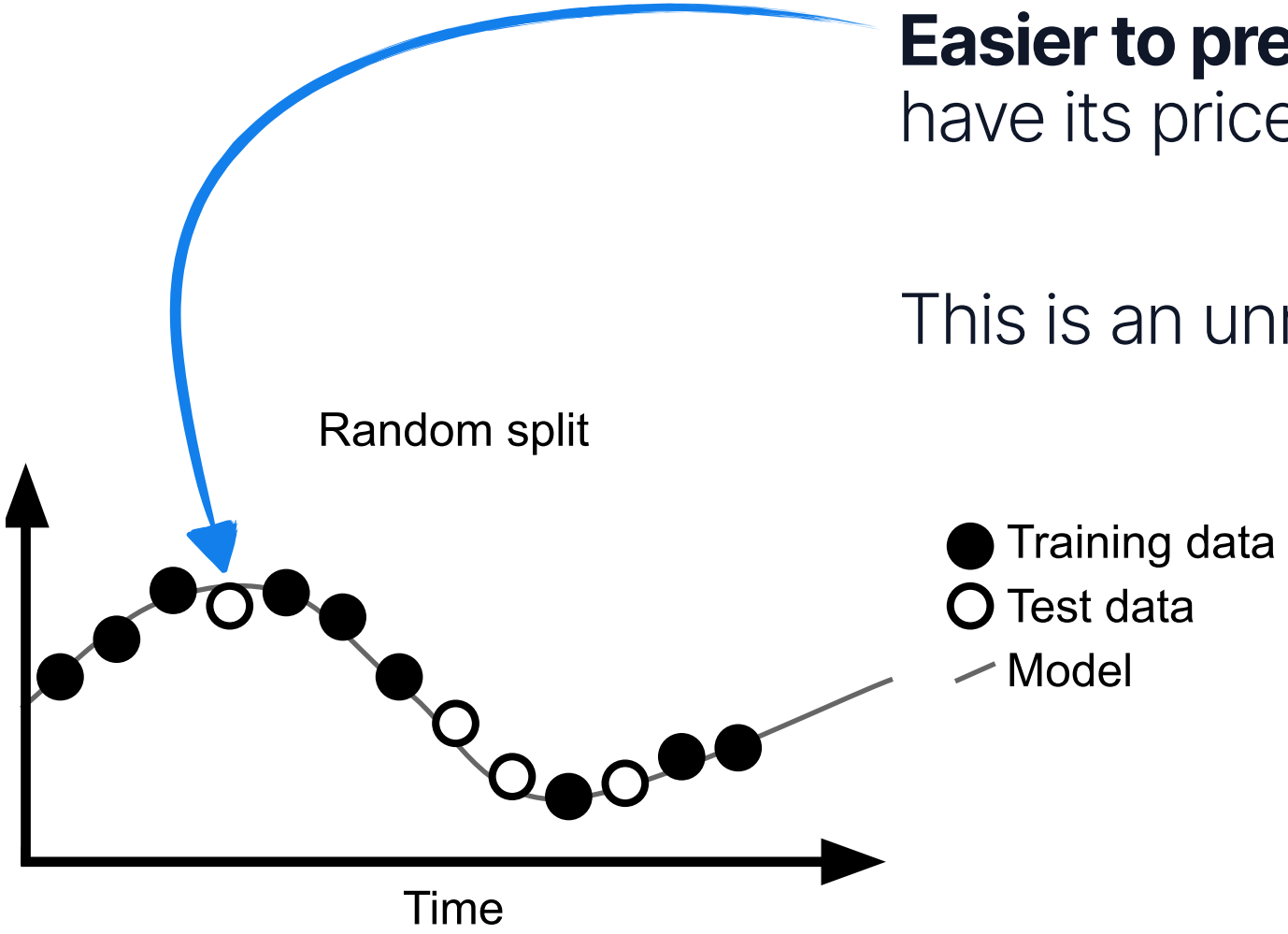
P5. Data [in]dependence

- You are training a ML model for predicting stock prices.
- Your data contains the stock price every hour.
- You randomly split training and test data.



What is the problem here?? 🤔

P5. Data [in]dependence



Easier to predict a stock price if you have its price before and after!

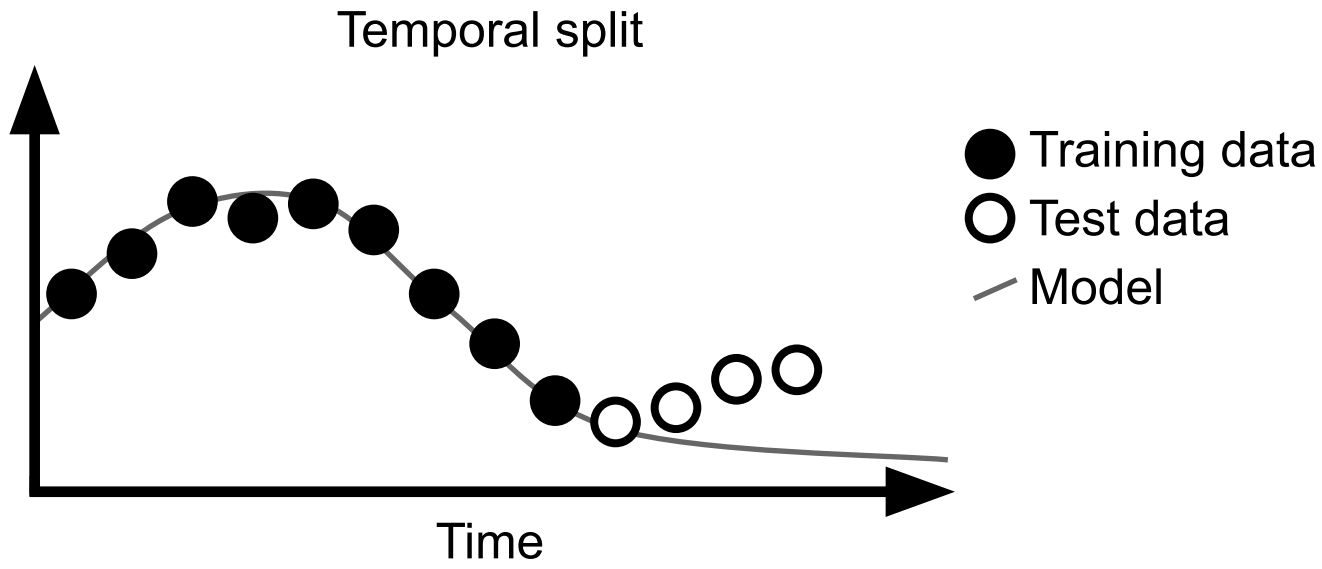
This is an unrealistic scenario.

P5. Data [in]dependence

We should adopt a data **split technique** that keeps data points independent and realistic.

For example, group dependent data, and perform random sampling over the clusters.

Can you think on other data dependency scenario? 🤔



P6. Label Leakage

and the tank urban legend

P6. Label Leakage

- Happens when the model detects unrelated information that, somehow, leads to the label.
- The model does not learn the task, it detects the data leak, instead.
- This shortcut does not generalize in practice.

P6. Label Leakage

"Why Should I Trust You?"
Explaining the Predictions of Any Classifier

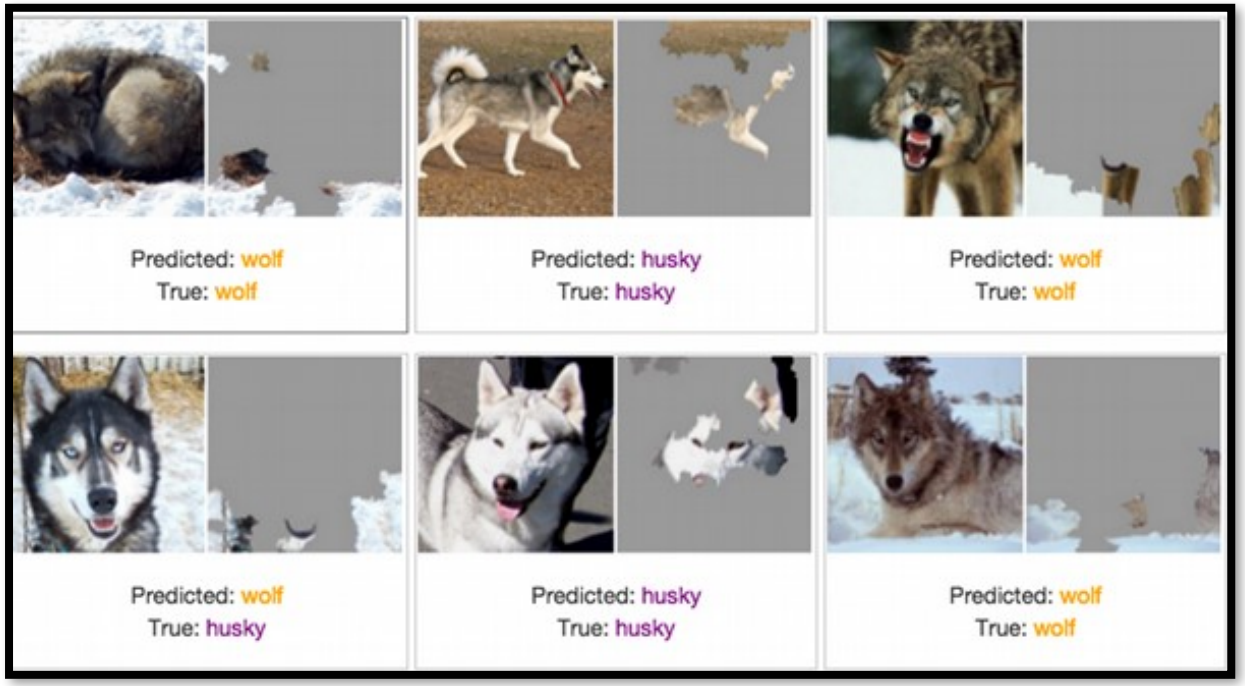
Marco Tulio Ribeiro
University of Washington
Seattle, WA 98105, USA
marcotcr@cs.uw.edu

Sameer Singh
University of Washington
Seattle, WA 98105, USA
sameer@cs.uw.edu

Carlos Guestrin
University of Washington
Seattle, WA 98105, USA
guestrin@cs.uw.edu

https://arxiv.org/pdf/1602.04938

The classifier predicts "Wolf" if there is snow (or light background at the bottom), and "Husky" otherwise



Going Beyond Accuracy

Accuracy Limitations

A model with 80% accuracy rate does not get 80% of its predictions right, **why?** 🤔

Accuracy Limitations

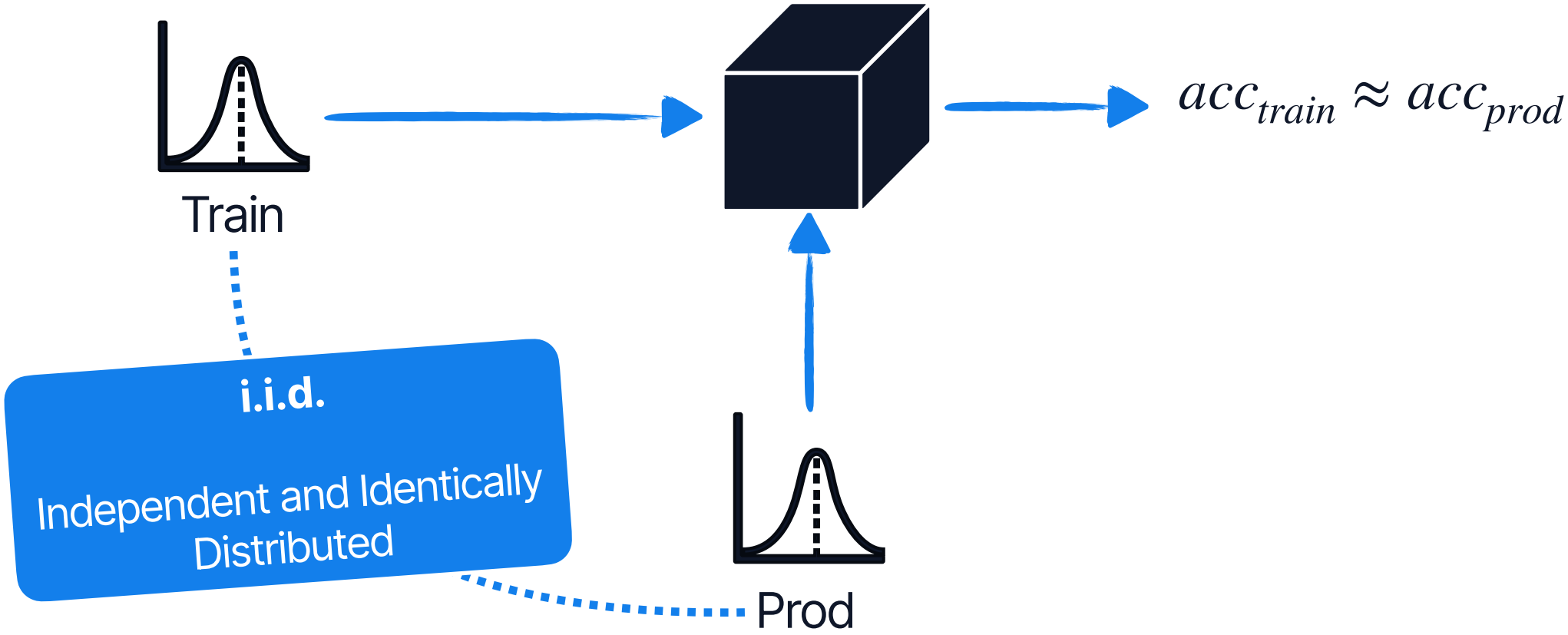
A model with 80% accuracy rate does not get 80% of its predictions right, **why?** 🤔

A. Accuracy describes the model fitness.

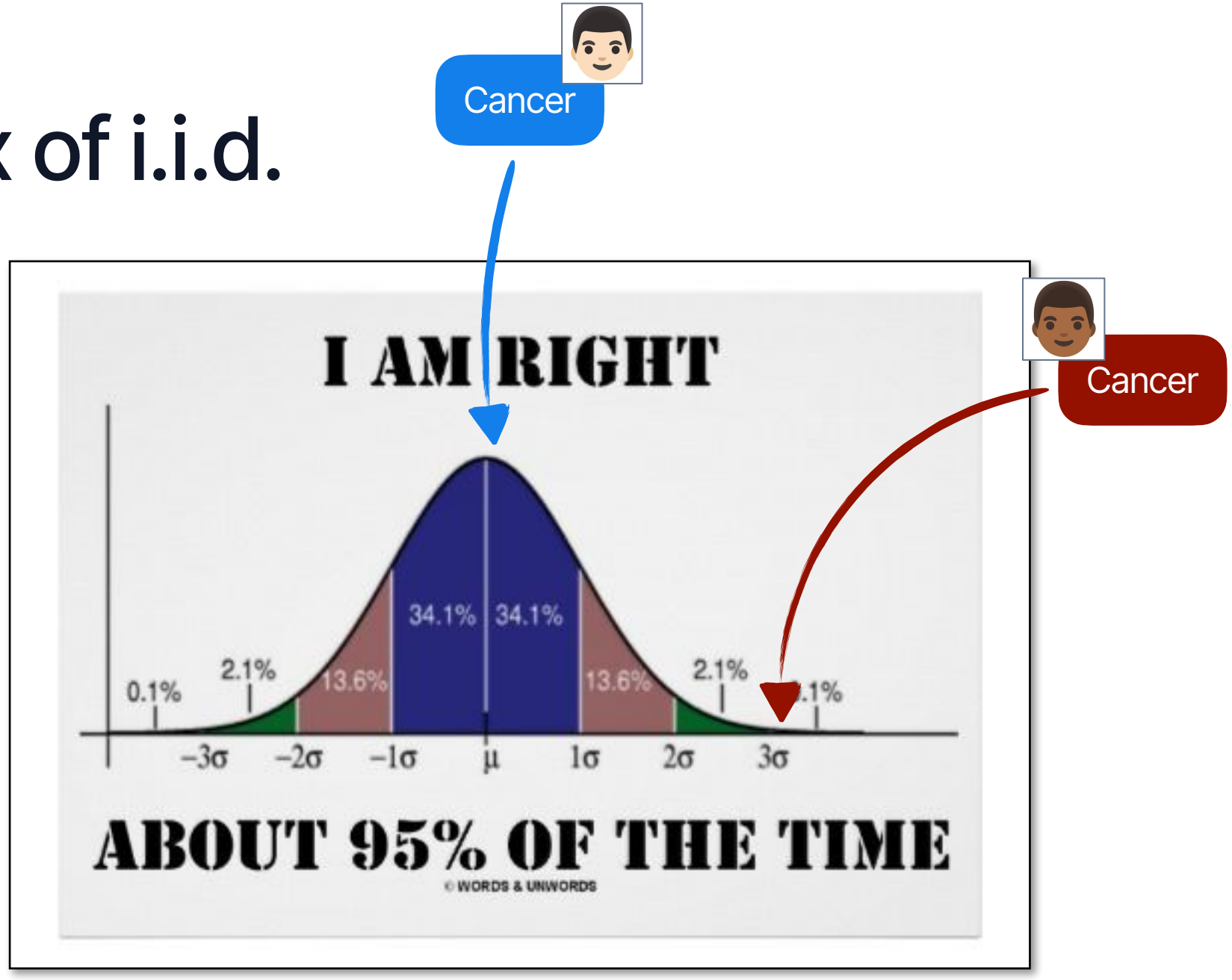
B. Model fits differently across the data distribution.

C. A single value blunt these particularities.

The paradox of i.i.d.

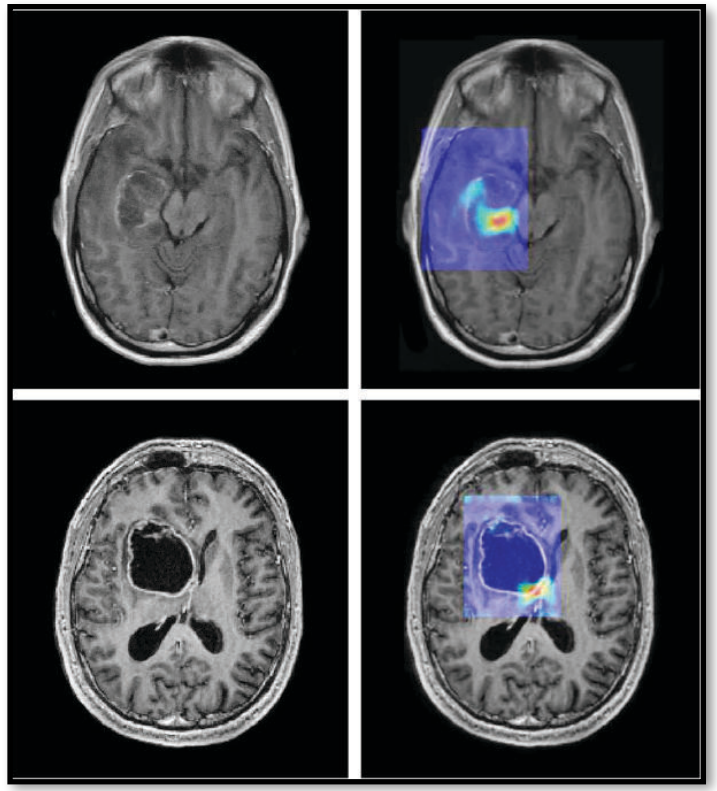


The paradox of i.i.d.



What if...

Not every input is equally important?



Consider a **cancer detector** system:

A. Which inputs are more important?

B. Which mistakes compromise product usage?

C. How frequently they are?

What if...

Not every input is equally important?

Consider a **voice assistant** tool:

A. Which inputs are more important?

B. Which mistakes compromise product usage?

C. How frequently they are?

What if...

Not every input is equally important?

- In many settings, some inputs are more important than others.
- Failure at such cases compromise product’s adoption.
- Important inputs can align with frequency (e.g., voice assistant), but does not need to (e.g., cancer detection).

What if...

Not every input is equally important?

How can we **identify** important inputs?

How can we **measure** model quality on them?

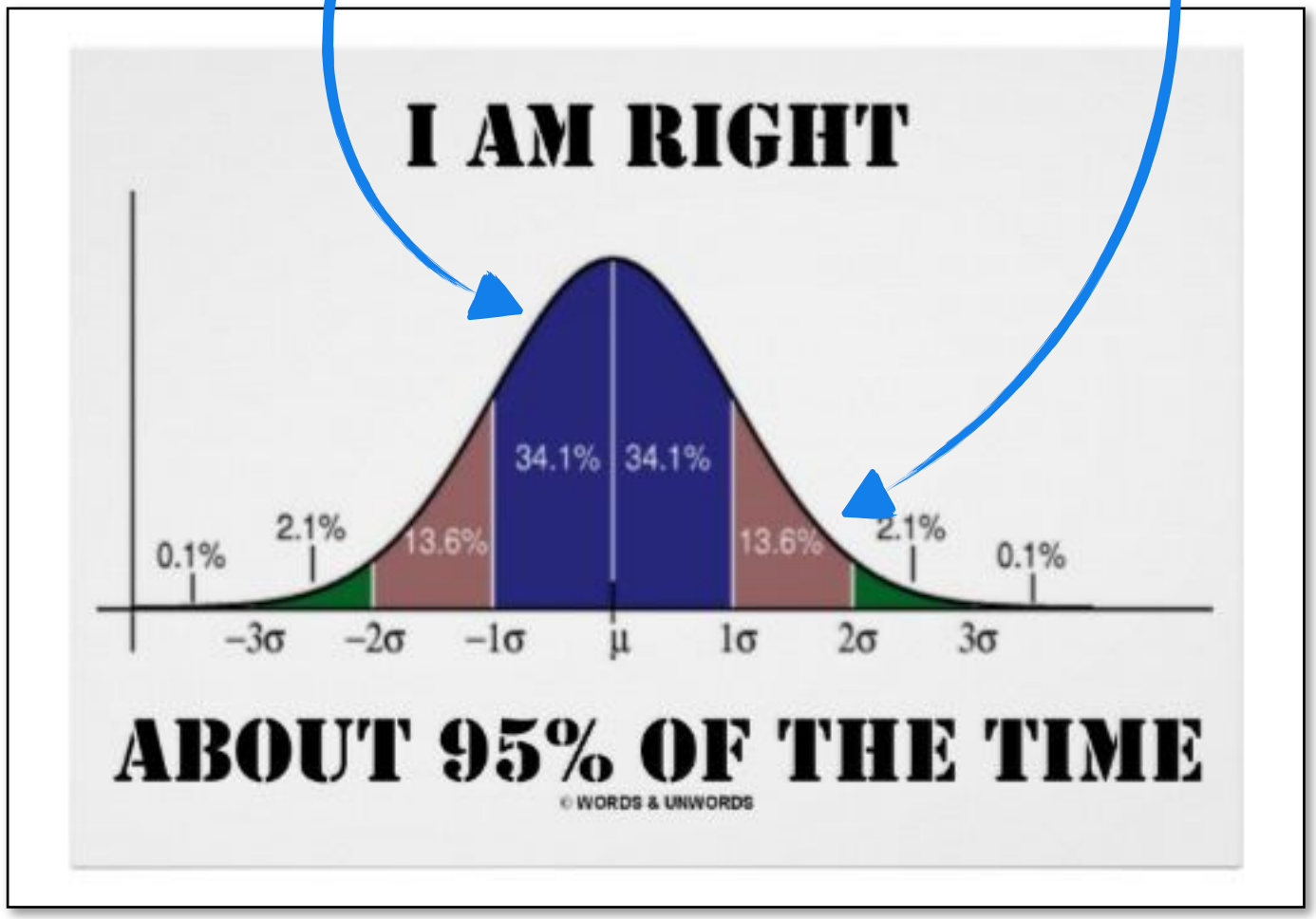
What if...

Some inputs are rare?

"Hey Siri, call mom"

"Hey Siri, call mom"

Your production model



What if...

Some inputs are rare?

- Models make systematic, not random, errors, which reflect patterns learned from training data.
- If rare inputs are also missed in test/validation, **such failure does not appear even in accuracy**.
- Such cases may be visible only in production.
- Consequence: unequal service quality for minority groups.

What if...

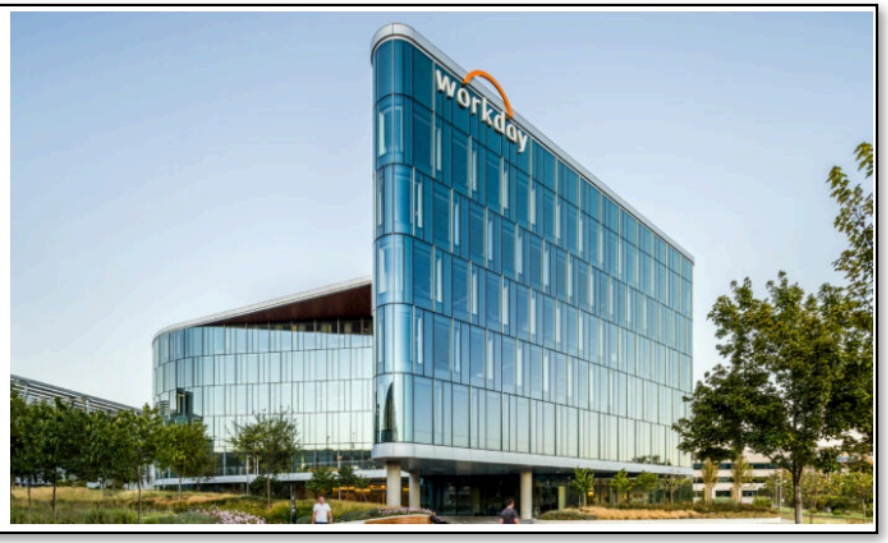
Some inputs are rare?

Landmark Workday case signals new AI hiring risk

Emerging HR Tech | HR Technology

By: **Jen Colletta** Date: **March 20, 2026**

Share post:



"Lead plaintiff Derek Mobley, who is Black, over 40, and has anxiety and depression, alleges **he was rejected from more than 100 jobs** [...], often **receiving automated rejection emails** within an hour of applying or in the middle of the night."

https://www.legalanalysis.org/workday-lawsuit

What if...

Is the model right for the wrong reason?

- Training/testing on i.i.d does not guarantee the model has learned the task.
- Quite the opposite, working with similar distributions can boost spurious correlations.
- Generally noticed as soon as the model goes to production.
- Solution: evaluate beyond i.i.d to increase model's robustness.
  By how much? 🤔

What if...

Is the model right for the wrong reason?

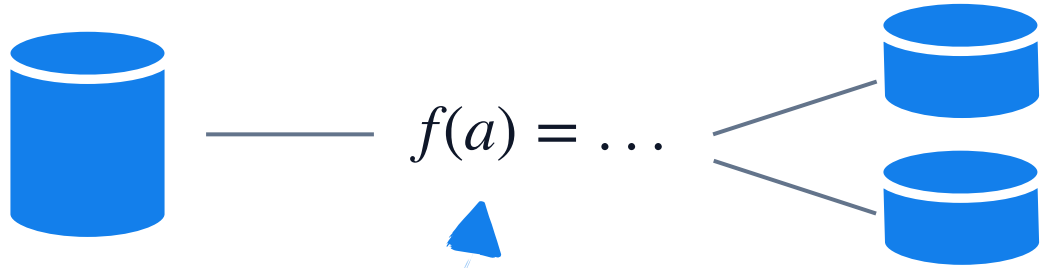
- Training/testing on i.i.d does not guarantee the model has learned the task.
- Quite the opposite, working with similar distributions can boost spurious correlations.
- Generally noticed as soon as the model goes to production.
- Solution: evaluate beyond i.i.d to increase model's robustness.
  By how much? 🤔

Slicing

Slicing

- Allow people to analyze model's accuracy for subsets of the test data.
- Testers intentionally slice the data according to their needs.
  - Ex: analyze cancer prognosis by age, by region, by hardware, etc.
- Goal: **detect discrepancies** between distinct slices.

Selecting Slices



- A **slicing function** cuts the data based on the features available for the test data.
- The **function** should be implemented based on our understanding of the problem.
  - Ex: create a slice for frequent voice commands, or specific type of aggressive cancers.
- For troubleshooting, hypotheses can guide slices thresholds.

Example: Slicing by cancer type

	Actually: Grade 5 cancer	Actually: Grade 3 cancer	Actually: Benign
Predicted: Grade 5 cancer	10	6	3
Predicted: Grade 3 cancer	3	24	10
Predicted: Benign	5	22	82

$$acc_{all} = \frac{10 + 24 + 82}{10 + 6 + 3 + 3 + 24 + 10 + 5 + 22 + 82} = 0.70$$

$$acc_{g5} = \frac{10 + 138}{10 + 9 + 8 + 138} = 0.90$$

$$acc_{g3} = \frac{24 + 100}{24 + 28 + 13 + 100} = 0.75$$

Creating Slices

- Easy mode: features already available, you just apply slicing function over them.
- More **useful** if we **create them based on hypotheses**, so we don't need to be stuck with feature slices, only.

Analyzing Slices

- **Goal: detect underperforming slices**
  - ⚠️ Small slices might be unreliable.
- Statistical tests may also help better detect them.
- Different fixes/mitigations can be applied to each slice.
  - Restrict inputs for unreliable corner cases
  - Design different failure modes for low confidence slices (prompt over automation).

Behavioral Testing

Behavioral Testing

- Curate test data to **test** specific **model capabilities**.
- **Behavior** $\Longrightarrow$ any characteristic we expect a model to have learned.
  - Ex: cancer model relies more on the shape of dark areas.
- Shortcuts $\Longrightarrow$ anti-behaviors, this test can ensure the model does not use them.

Detecting Behaviors

- Behaviors are derived from requirements.
- As specifications are not clear, we can not identify behaviors upfront.
- A checklist of common concepts a model must learn.

Capabilities	Descriptions
Vocab/POS	important words or word types for the task.
Named entities	appropriately understanding named entities.
Nagation	understand the negation words.
Taxonomy	synonyms, antonyms, etc.
Robustness	to typos, irrelevant changes, etc.
Coreference	resolve ambiguous pronouns, etc.
Fairness	not biasing towards certain gender/race groups.
Semantic Role Labeling	understanding roles such as agent, object, etc.
Logic	handle symmetry, consistency, and conjunctions.
Temporal	understand order of events.

M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh. "Beyond accuracy: behavioral testing of NLP models with CheckList". ACL, 2020.

Detecting Behaviors

Labels: positive, negative, or neutral; INV: same pred. (INV) after removals/ additions; DIR: sentiment should not decrease (↑) or increase (↓)

	Test *TYPE* and Description	Failure Rate (%)					Example test cases & expected behavior
						RoB	
Vocab.+POS	***MFT:*** Short sentences with neutral adjectives and nouns	0.0	7.6	4.8	94.6	81.8	The company is Australian. neutral That is a private aircraft. neutral
	MFT: Short sentences with sentiment-laden adjectives	4.0	15.0	2.8	0.0	0.2	That cabin crew is extraordinary. pos I despised that aircraft. neg
	INV: Replace neutral words with other neutral words	9.4	16.2	12.4	10.2	10.2	@Virgin should I be concerned that → when I'm about to fly ... INV @united the → our nightmare continues... INV
	DIR: Add positive phrases, fails if sent. goes down by > 0.1	12.6	12.4	1.4	0.2	10.2	@SouthwestAir Great trip on 2672 yesterday... You are extraordinary. ↑ @AmericanAir AA45 ... JFK to LAS. You are brilliant. ↑
	DIR: Add negative phrases, fails if sent. goes up by > 0.1	0.8	34.6	5.0	0.0	13.2	@USAirways your service sucks. You are lame. ↓ @JetBlue all day. I abhor you. ↓
Robust.	***INV:*** Add randomly generated URLs and handles to tweets	9.6	13.4	24.8	11.4	7.4	@JetBlue that selfie was extreme. @pi9QDK INV @united stuck because staff took a break? Not happy 1K.... https://t.co/PWK1jb INV
	INV: Swap one character with its neighbor (typo)	5.6	10.2	10.4	5.2	3.8	@JetBlue → @JeBtlue I cri INV @SouthwestAir no thanks → thakns INV
NER	***INV:*** Switching locations should not change predictions	7.0	20.8	14.8	7.6	6.4	@JetBlue I want you guys to be the first to fly to # Cuba → Canada... INV @VirginAmerica I miss the #nerdbird in San Jose → Denver INV
	INV: Switching person names should not change predictions	2.4	15.1	9.1	6.6	2.4	...Airport agents were horrendous. Sharon → Erin was your saviour INV @united 8602947, Jon → Sean at http://t.co/58tuTgli0D, thanks. INV

M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh.
"Beyond accuracy: behavioral testing of NLP models with CheckList". ACL, 2020.

Detecting Behaviors

In practice, behaviors are often derived from error analysis. They often works with boundary decisions.

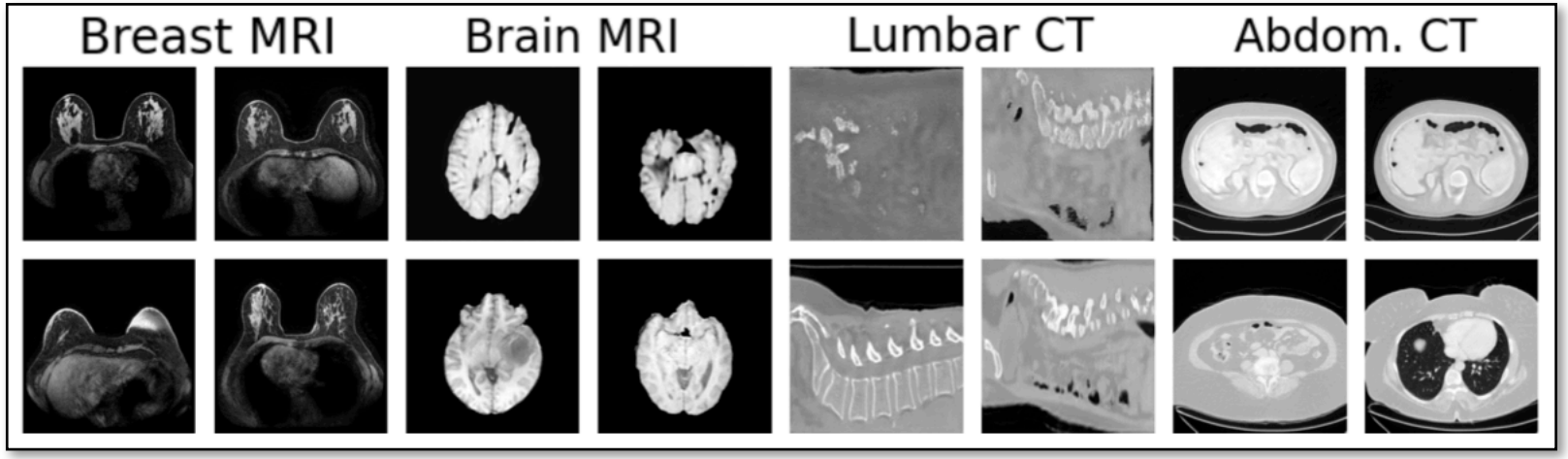
- ☑ Have good understanding of the problem to map corner cases (linguistic concepts in NLP).
- ☑ Observe common mistakes (cancer model using shortcuts).
- ☑ Observe how humans perform the task (radiologists analyzing false positives).

Where to obtain test data?

Domain-specific generators: generate tests from patterns.

A. Testing negation in sentiment analysis.
   "I [NEGATION] [POS_VERB] the [THING]"

B. Generate realistic renderings of cancer images.



Where to obtain test data?

New test data from scratch that tests a specific behavior.

A. Crowdsourcing new test data.

https://gor-grigoryan.medium.com/how-recaptcha-turned-internet-users-into-unpaid-ai-trainers-a2107adf31e3

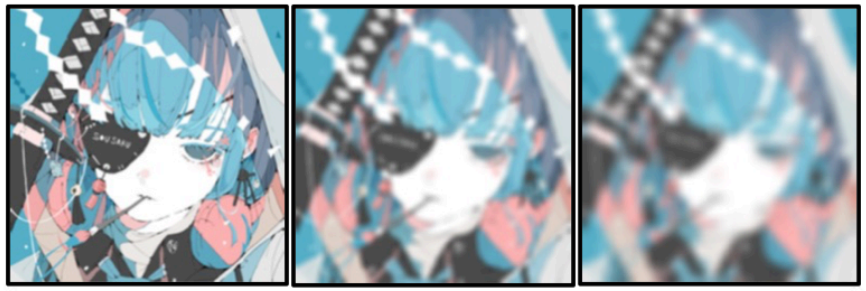
B. Collect data from populations beyond current distribution.

Where to obtain test data?

Transformations of the existing data to introduce variations relying on certain behaviors.

A. Replacing words with synonyms for sentiment analysis
   *"@AmericanAir **[thank you → many thanks]** we got on a different flight to Chicago."*

B. Image effects for object detection



Testing Unlabeled Data

Testing Unlabeled Data

- Most **testing requires labels** to determine whether predictions are correct.
- However, **labeling** test data **can be expensive**, while unlabeled data is abundantly available.
- Idea: focus on evaluating **how model prediction reacts to data with different but related properties**.

Invariants and Metamorphic Relations

- Some properties can be expressed as invariants, thus can be tested without need to know the expected output.
- **Invariant: a property which holds constant** across different predictions.
  - Ex: A summarizer should always generate a shorter text.
- In ML, most useful invariants describe the relative behavior between distinct inputs; a.k.a metamorphic relation.

Examples

A. The cancer prognosis model m should predict the same outcome for both the original and slightly cropped one.
$\forall x \,.\, m(x) = m(crop(x))$

B. A credit rating model m should not depend on gender.
$\forall x \,.\, m(x) = m(swap\_gender(x))$

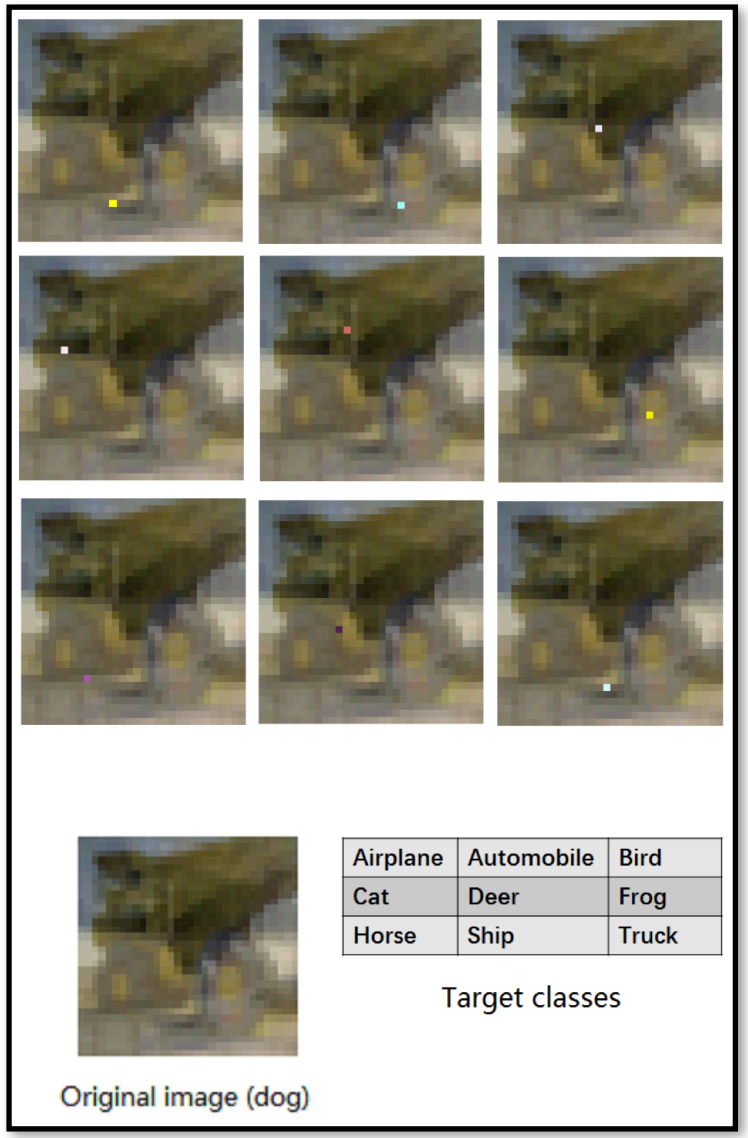
C. If two costumers differ only in income, a credit limit modem m should never suggest a lower credit limit for the one with the highest income.
$\forall x \,.\, m(x) \leq m(highest\_income(x))$

Examples

"In fact, by increasing the number of pixels up to five, a considerable number of images can be simultaneously perturbed to eight target classes. In some rare cases, **an image can go to all other target classes with one-pixel modification**."

J. Su, D. V. Vargas, and K. Sakurai. One pixel attack for fooling deep neural networks. IEEE Trans. Evol. Comput., 2019.

What are we testing, after all? 🤔



Identifying Invariants

Requires a deep understanding of the problem, but a few use cases are common.

A. Most invariants aim at testing model robustness.

B. Invariants to things the model should specifically not rely on.
   ex: cancer prediction should be the same for images with and without scanner hardware info in the top-right corner

C. Domain-specific expectations that can be encoding.

Approaches for Invariant Testing

👍🏽 Don’t reject a model for a single mistake. Most invariants are not strict.

A. Measure the relative frequency of invariant violations over production data.

B. Generate data that somehow resembles production data (i.e., avoid random ones). Domain-specific generators and GANs are good options.

C. Formal methods to prove an invariant for a model holds for all possible inputs.

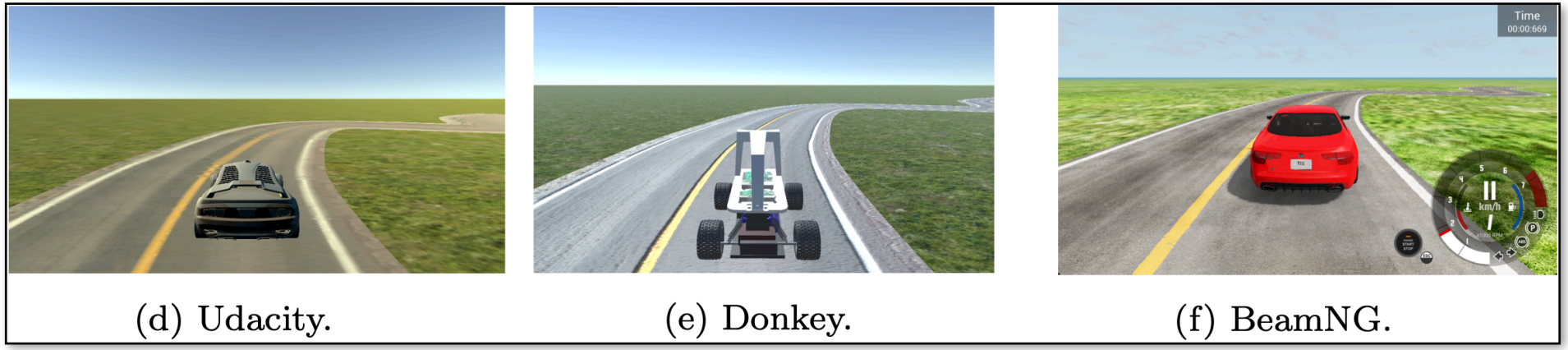
Other Approaches

Differential Testing

- Compares model’s prediction against a reference implementation.
- We can also compare multiple trained models and use majority voting to assume correct label.
- Not commonly used, as we typically lack reference implementations.

Simulation-based Testing

- Use it if it easier to generate inputs from real outputs.



(d) Udacity. (e) Donkey. (f) BeamNG.

L. Sorokin, M. Biagiola, and A. Stocco, “Simulator ensembles for trustworthy autonomous driving systems testing,” *EMSE*. 2026.

- Expensive to build and hard to implement; counter factual can be hard to leverage.
  ex: How to insert cancer in a healthy person?

Simulation-based Testing

Does not work if:

- The input-output is unknown, making it impossible to simulate.
  ex: predicts the time to remission for cancer.
- The reverse computation (output → input) is as hard as the computational to be evaluated.
  ex: generate an image of a “car" to an object detector.

joao@dcc.ufmg.br